

A

QUESTION 1

A. Fill in the blanks. Each blank is worth 2 points. (10P)

- A human-readable, code-like description of an algorithm is called
- The first method executed in a C# console program is
- In C#, $12 / 5$ gives.....and $12 \% 5$ gives
- A variable declared with `const` cannot beafter its value is assigned.

B. Write the exact console output produced by the following code, one line at a time, in the table below. (10P)

```
int a = 8, b = 3;
int c = a++ + ++b;
bool ok = (c >= 12) && (a % b == 1);
Console.WriteLine(a);
Console.WriteLine(b);
Console.WriteLine(c);
Console.WriteLine(ok);
if (ok)
    Console.WriteLine("Release");
else
    Console.WriteLine("Hold");
```

Line 1	
Line 2	
Line 3	
Line 4	
Line 5	

C. Write pseudocode or C# code for payroll calculation. Read **hoursWorked** and **hourlyWage**. If **hoursWorked** < 0 or **hourlyWage** <= 0, print **ERROR**. If **hoursWorked** <= 40, pay is **hoursWorked * hourlyWage**. Otherwise, pay is **40 * hourlyWage + (hoursWorked - 40) * hourlyWage * 1.5**. Print only the final pay or **ERROR**. (10P)

B

QUESTION 1

A. Fill in the blanks. Each blank is worth 2 points. (10P)

- A human-readable, code-like description of an algorithm is called
- The first method executed in a C# console program is
- In C#, $9 / 5$ gives and $9 \% 5$ gives
- A variable declared with `const` cannot be after its value is assigned.

B. Write the exact console output produced by the following code, one line at a time, in the table below. (10P)

```
int a = 8, b = 3;
int c = a++ + ++b;
bool ok = (c >= 12) && (a % b == 1);
Console.WriteLine(a);
Console.WriteLine(b);
Console.WriteLine(c);
Console.WriteLine(ok);
if (ok)
    Console.WriteLine("Release");
else
    Console.WriteLine("Hold");
```

Line 1	
Line 2	
Line 3	
Line 4	
Line 5	

C. Write pseudocode or C# code for payroll calculation. Read **hoursWorked** and **hourlyWage**. If **hoursWorked** < 0 or **hourlyWage** <= 0, print ERROR. If **hoursWorked** <= 50, pay is **hoursWorked * hourlyWage**. Otherwise, pay is **50 * hourlyWage + (hoursWorked - 50) * hourlyWage * 1.5**. Print only the final pay or ERROR. (10P)

QUESTION 2

- A. Trace the following switch statement and write the exact console output. (10P)

```
using System;
public class HelloWorld {
    public static void Main(string[] args) {

        int action = 3;
        int value = 10;
        switch (action) {
            case 1:
                value += 5;
                break;
            case 2:
                value -= 3;
                break;
            case 3:
                value *= 2;
                goto case 2;
            default:
                value = 0;
                break;
        }
        Console.WriteLine(value);
    }
}
```

OUTPUT

- B. The following program reads five machine cycle times from user input, counts how many of them exceed a specified target value, and then displays the result. Complete the missing code. Note: Assume that the required libraries and the Main() function have already been provided; no additional imports are necessary. (20P)

```
1 int[] cycleTimes = new int[5];
2 int target = 10;
3 int aboveTarget = 0;
4
5 //Read user input 5 times
6 for ( _____ ) {
7     cycleTimes[i] = Convert.ToInt32( _____ );
8 }
9
10 //Count Num of Target
11 foreach( _____ )
12 {
13     if ( _____ )
14         aboveTarget++;
15 }
16
17 //Print
18 Console.WriteLine(aboveTarget);
```

QUESTION 3: Write a working, single-file C# console program, including all necessary `using` directives, function definitions, and class definition. Your program should ask the user to enter the name of an input file (e.g: `input.txt`) . The input file contains one integer per line, where each integer represents the number of inspected units for one day. Create a function called `ReadValues` that receives the file name as a parameter. `ReadValues` function should use a `try/catch/finally` block to read all lines from the file, convert them to integers, and return the values as an integer array. Catch any `FileNotFoundException`, `FormatException`, and `IOException` and print an appropriate error message and terminate the program. In the `Main` method, process the array returned by `ReadValues`. Check and ignore any negative values, count the number of valid entries, and calculate the total number of inspected units. Create another function called `SaveResult` that receives the valid-day count and total as parameters. `SaveResult` function should append one line to `quality_summary.txt` in the following format: `ValidDays=3 Total=450` . Use append mode; assume the output file can be written successfully. Finally, print a message to the user showing the valid-day count and total inspected units. **(40P)**